

Filtering And Sorting

1 What is Filtering?

In backend architecture, **filtering is data selection**.

It answers:

From a large dataset, which records satisfy a condition?

This usually happens:

- In a database
- In a search engine (Elastic)
- In a distributed system
- In an analytics engine

Example conceptually:

You have 10 million users.

You want:

- Users from India
- Age > 25
- Is premium = true

That is filtering.

What Actually Happens Internally (Database Perspective)

Suppose you're using PostgreSQL.

When you run:

```
SELECT * FROM users WHERE age > 25 AND country = 'India';
```

The database does NOT just “loop through everything” (unless it has to).

It chooses a strategy:

Case 1: No Index → Full Table Scan

Database checks every row.

Time complexity: $O(n)$

This is expensive for millions of records.

Case 2: With Index → Index Scan

If there's an index:

```
CREATE INDEX idx_users_country_age ON users(country, age);
```

Now database:

1. Looks inside B-Tree index
2. Finds matching range quickly
3. Fetches only matching rows

Time complexity becomes roughly $O(\log n)$

That is massive performance improvement.

2 What Sorting Means in Backend Systems

Sorting in backend is:

Reordering result set according to some field.

Example:

```
SELECT * FROM users  
WHERE country = 'India'  
ORDER BY created_at DESC;
```

Sorting is much heavier than filtering.

Why?

Because sorting requires:

- Comparing many rows
- Possibly loading rows into memory
- Sometimes using temporary disk space

3 Sorting Internals (Database View)

Databases use algorithms like:

- External Merge Sort

- QuickSort (in-memory)
- Heap sort (sometimes)
- Index-based ordering

Case A: No Suitable Index

Database:

1. Applies filter
2. Loads results
3. Sorts them
4. Returns

If result set is large:

- It may spill to disk
- Performance drops drastically

Case B: Using Index for Sorting

If query matches index order:

```
CREATE INDEX idx_users_created_at ON users(created_at DESC);
```

Then:

```
SELECT * FROM users ORDER BY created_at DESC;
```

Database can:

- Read index in sorted order
- No extra sorting needed

This is called **index-only ordering**

Much faster.

Filtering usually happens before sorting.

Why?

Because sorting fewer rows is cheaper.

Example execution plan:

1. Index Scan (filter)
2. Sort (on reduced dataset)

10 When Systems Break

Common backend mistake:

- Apply sorting on large non-indexed column
- Filter after fetching all rows into app memory
- Sort inside application server

Example of bad design:

```
const users = await db.getAllUsers();  
return users.sort(...)
```

This destroys scalability.

Always push filtering + sorting to database layer.

1 1 Mental Model for Backend Engineers

Filtering = Data Reduction Strategy

Sorting = Data Arrangement Strategy

Filtering improves performance.

Sorting often costs performance.

Good backend engineers:

- Design indexes based on filtering patterns

- Align sorting fields with index order
- Use composite indexes
- Avoid unnecessary ORDER BY

If your application:

- Frequently filters by country
- Frequently sorts by created_at

Best index:

```
CREATE INDEX idx_country_created  
ON users(country, created_at DESC);
```

Why this order?

Because:

- First column handles filtering
- Second handles sorting
- Both operations satisfied by same index

That is backend optimization thinking.

1. What is the difference between filtering and sorting?

Expected depth:

- Filtering reduces dataset.
- Sorting reorders dataset.
- Filtering usually happens before sorting in execution plan.
- Sorting is more expensive.

2. What is the time complexity of filtering vs sorting?

Expected:

- Filtering: $O(n)$ (or $O(\log n)$ with index)

- Sorting: $O(n \log n)$
- External sort if memory limit exceeded

3. Why is ORDER BY slow sometimes?

Answer:

- No suitable index
- Requires filesort (in MySQL)
- Sorting large result set
- Disk-based sorting